Control Dataset -

Testing Discretization Invariance via 1D Integration

Fundamental Research in Machine and Deep Learning

Leonardo Castello (6553583)

1. Introduction and Hypothesis

Traditional deep learning architectures are designed to map inputs between finite-dimensional spaces. Therefore, when they are applied to physical systems, the continuous space must be discretized into a fixed grid of points before one of these models can process it.

The main consequence of this is that learnable parameters of the models are tied to the specific grid resolution used during training. Therefore, a conventional model cannot naturally process inputs or predict outputs at higher resolutions compared to the one used during training. Testing the model on a different grid resolution causes severe accuracy drops (or total failure), mandating structural modification or retraining.

To solve this problem, Kovachki et al. (2022) proposed Neural Operators in their paper *Neural Operator: Learning Maps Between Function Spaces with Applications to PDEs* [1]. Instead of mapping fixed-size vectors, Neural Operators map directly between infinite-dimensional spaces. This shift in paradigm, makes them discretization invariant, which means that the model parameters remain the same across any sampling resolution.

This blog post introduces a custom control dataset designed to test this exact property. The specific hypothesis under test is: *'A standard vector-to-vector model will show significant accuracy drops when tested at a different resolution from the training baseline. On the other hand, a discretization-invariant model will maintain a constant performance across all grid sizes*.'

Verifying such property is highly motivated by engineering needs. In scientific computing, high resolution meshes are required to capture fine-scale details, and a discretization-invariant model would mean unlocking what's called zero-shot super-resolution [2]. That is the ability to train an architecture only on cheap, low-resolution data and successfully evaluate it on fine grids without the need for retraining. This capability was popularized by Li et al. (2021) in their paper *Fourier Neural Operator for Parametric Partial Differential Equations* [2].

2. Dataset Design: The 1D Antiderivative

To evaluate discretization invariance, it's important that the resolution is the only variable that changes in the environment. Often, standard datasets for partial differential equations rely on numerical solvers to compute output labels. However, these solvers introduce approximations and errors that might vary depending on the grid resolution. This means that even if a performance drop is observed, it is hard to know whether the model's architecture failed or if the label itself changed in accuracy.

To eliminate this issue, a toy problem with an analytical was used: the 1D Antiderivative operator, which maps an input function $a(x)$ to an output function $u(x)$ by integrating over the domain X (initial condition: $u(0) = 0$):

$$u(x) = \int_0^X a(s)ds$$

The functions were generated as a random combination of band-limited sine waves ($K = 8$):

$$a(x) = \sum_{k=1}^{K} c_k \sin(2\pi k x + \phi_k)$$

Since the integral of sine waves is known analytically, the exact true output function $u(x)$ can be easily calculated:

$$u(x) = \sum_{k=1}^{K} \frac{c_k}{2\pi k} (\cos(\phi_k) - \cos(2\pi k x + \phi_k))$$

This means that $a(x)$ and $u(x)$ can be evaluated at any arbitrary grid resolution with mathematical exactness and no solver error. Also, this antiderivative operator is a recognized operator benchmark that was initially used by Lu et al. (2021) in their *DeepONet* paper to verify function mapping capabilities [3].

3. Visual Examples and Intuition

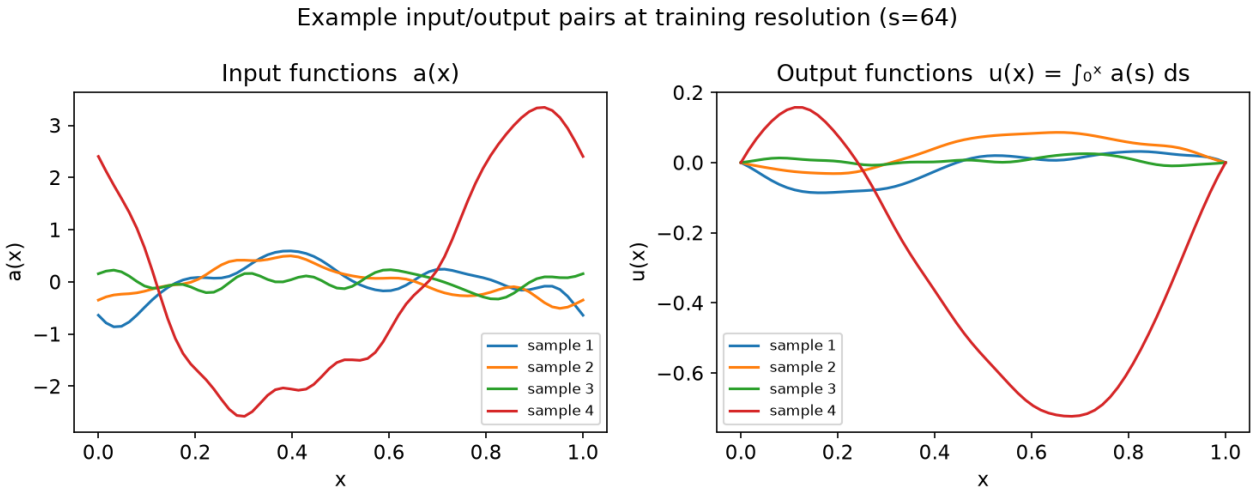

Figure 1: Input-output pairs ($a(x)$ and $u(x)$) from the control dataset, evaluated at the training resolution ($s = 64$)

Figure 1 illustrates the mapping mechanism. It shows random input functions $a(x)$ alongside their analytical counterparts $u(x)$, evaluated at the standard training resolution of $s = 64$ spatial points.

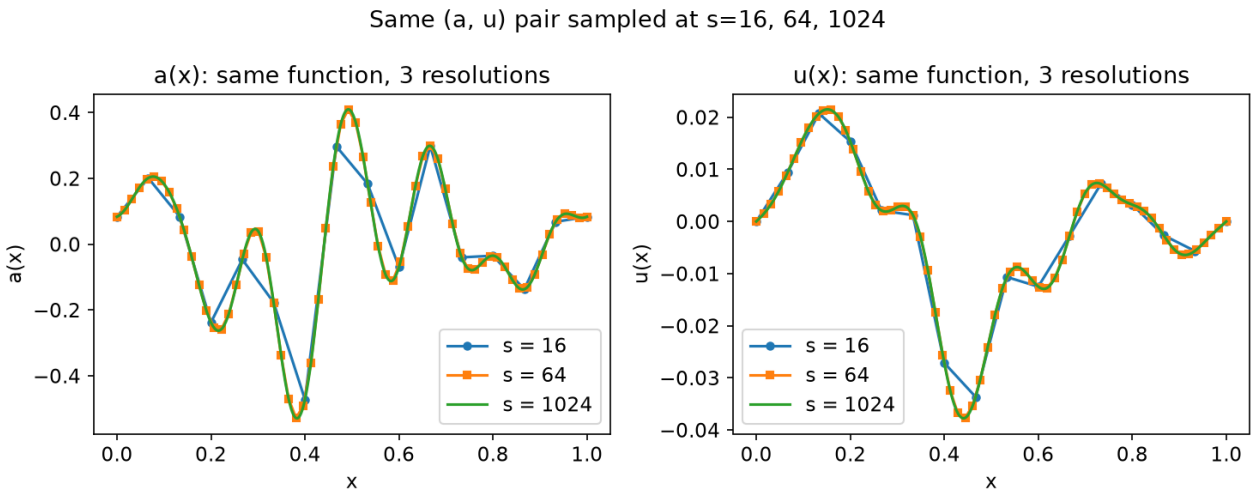

Figure 2: Sampling at different resolutions $s = [16, 64, 1024]$ of a single function pair, showing exact mathematical alignment

Figure 2 demonstrates the control mechanism. It maps a single function sampled at three different testing resolutions: a coarse grid ($s = 16$), the training grid ($s = 64$) and a very fine grid ($s = 1024$). Because an analytical solution was used, the function values do not shift or drift when the resolution changes.

4. Generation Methodology

This section describes how the dataset was generated:

- The training data consists of 1000 unique function pairs evaluated at the fixed resolution of $s = 64$. The testing data consists of 200 separate function pairs. To test the effects of discretization, the same 200 functions were evaluated at seven different spatial resolutions $s = [16, 32, 64, 128, 256, 512, 1024]$
- To ensure no leakage between splits, the training set was generated using a dedicated seed ($seed = 42$), while the testing set was generated using a different seed ($seed = 2024$).
- The script creates a uniform grid over the domain $[0,1]$. Each random input function $a(x)$, it draws a random sequence of wave coefficients for each of the $K = 8$ frequency components. Furthermore, random phase shifts $\phi_k \in [0,2\pi]$ were introduced. This ensures that every curve is unique.
- A verification script was used to validate the dataset:
    - Boundary alignment: $u(0) = 0$ across all functions and resolutions.
    - Grid consistency: Shared grid points ($x = 0$ and $x = 1$) gives the same results across all files
    - No data leakage: Training and testing coefficient matrices have no overlapping configurations
    - Calculus verification: $\frac{du}{dx}$ recovers $a(x)$

5. Evaluation Protocol

This final section describes the workflow required to test the hypothesis using the generated control dataset.

To begin, a researcher should select a standard ‘grid-dependent' model (such as a standard MLP) and a discretization-invariant one (such as a Fourier Neural Operator [2]). Both architectures must be trained exclusively on the training file containing the 1000 function pairs at the fixed resolution of $s = 64$. Also, to ensure fair comparison, both models should be optimized using identical loss function and parameters.

After training, the learned parameters should be frozen. Both models should then be evaluated without any modifications across the seven independent testing data (one file per resolution: $s = [16, 32, 64, 128, 256, 512, 1024]$).

Lastly, to verify the core hypothesis, the performance of both models should be displayed mapping the Test Error ($Y - axis$) against the Grid Resolution ($X - axis$):

- Grid-dependent baseline: The error curve is expected to form a V-shape. The network will achieve the lowest error rate at $s = 64$. However, as the testing resolution moves away from this baseline, the error should keep increasing.
- Neural Operator: The error curve is expected to remain flat (and low) across all testing resolutions, proving the discretization invariance of such model.

6. Code & Dataset Repositories

All Python scripts that were used, as well as the generated dataset, are available on GitHub:

https://github.com/leonardo-castello/1d-antiderivative-operator-dataset

References

[1] Kovachki, N., Li, Z., Liu, B., Azizzadenesheli, K., Bhattacharya, K., Stuart, A., & Anandkumar, A. (2022). Neural Operator: Learning Maps Between Function Spaces with Applications to PDEs. *Journal of Machine Learning Research*.

[2] Li, Z., Kovachki, N. B., Azizzadenesheli, K., Liu, B., Bhattacharya, K., Stuart, A., & Anandkumar, A. (2021). Fourier Neural Operator for Parametric Partial Differential Equations. In *9th International Conference on Learning Representations (ICLR 2021)*.

[3] Lu, L., Jin, P., Pang, Guofei., Zhang, Z., & Karniadakis, G. E. (2021). Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators.*Nature Machine Intelligence*.

AI & LLM Usage

Large Language Models were used as tools throughout the project to:

- Refine language: Edit, restructure, and improve the flow of the blog post.
- Code Troubleshooting: Analyse errors, debug, and resolve implementation issues.
- Algorithmic Guidance: Provide alternative perspectives on script organization and software optimization strategies.